unitTransformDist: A Distribution-Agnostic Toolkit for Unit-Interval Models via the Odds-Inverting Transform

by Mohamad Yamen Al Mohamad, Ali Akbar Heydari and Hossein Jabbari Khamnei

Abstract Data restricted to the open interval $(0, 1)$ – proportions, rates, and probabilities – are commonly modelled in R with **betareg** (beta regression), **simplexreg** (simplex regression), or the recently released **gkwdist** (generalized Kumaraswamy family). Each of these packages targets one specific parametric family. This paper introduces **unitTransformDist**, which instead constructs a $(0, 1)$-valued distribution from an arbitrary base distribution on $(0, \infty)$ via the odds-inverting transform $Y = 1/(1 + X)$, taking the base distribution's density, CDF, quantile, and random-generation functions directly as arguments rather than selecting from a fixed registry. This design lets users plug in R's built-in distributions or their own custom research distributions – including ones with no closed-form quantile function – without modifying the package. Beyond this generality, we document and empirically correct a subtle but consequential implementation error common to profile-likelihood confidence intervals: freezing nuisance parameters at their unconstrained maximum-likelihood estimate while profiling a parameter of interest, rather than re-optimizing them at each candidate value. A reproducible Monte Carlo study included in the package shows this mistake holds actual 95% coverage near 54–61% across sample sizes $n \in \{30, 60, 100, 200\}$, essentially independent of n, while the corrected procedure restores coverage to 90–94%. The package also provides Monte Carlo utilities for studying estimator bias and several bias-correction methods (jackknife, bootstrap, Firth-type), an experimental analytic bias-correction routine that we explicitly flag as unverified, and a small suite of diagnostics. We describe the package's design, its relationship to existing CRAN alternatives, its performance characteristics, and its current limitations, reporting negative and inconclusive results alongside positive ones.

1 Introduction

Continuous data confined to the open unit interval $(0, 1)$ arise throughout applied statistics: disease prevalence, vote shares, market shares, budget allocations, normalized indices such as the Gini coefficient, and proportions of any kind are all naturally modelled as values strictly between zero and one. R has a mature ecosystem for this class of problems. **betareg** implements beta regression, linking the mean and precision of a beta-distributed response to covariates through a GLM-style linear predictor (Cribari-Neto and Zeileis, 2010); a more recent extension supports an extended-support beta distribution that accommodates boundary observations at exactly 0 or 1. **simplexreg** instead models the response as simplex-distributed, following Song and Tan (2000) and implemented for R by Zhang et al. (2016), with maximum likelihood and generalized estimating equations for longitudinal data. **gkwdist** implements the seven-member generalized Kumaraswamy family (Carrasco et al., 2010), nesting the ordinary Kumaraswamy and beta distributions as special cases, with a full C++ backend via **RcppArmadillo** for speed and analytical derivatives for numerical stability. **logitnorm** provides density, distribution, quantile, and random-generation functions together with moment-based estimation for the logit-normal distribution. The **MLE** package on CRAN fits a menu of named unit-interval distributions – beta, logit-normal, unit-Weibull, standard power, inflated beta, hyper-secant, Kumaraswamy, simplex, zero-inflated logit-normal, continuous Bernoulli, and a logistic-beta variant – via maximum likelihood or Newton–Raphson.

A structurally different idea appears in **Newdistns**, which takes an arbitrary baseline

CDF G on any support and layers a fixed menu of generator families on top of it: Marshall–Olkin G distributions, exponentiated G distributions, beta G distributions, Weibull G distributions, log-gamma G distributions of two kinds, exponentiated generalized G distributions, exponentiated Kumaraswamy G distributions, geometric exponential Poisson G distributions, truncated-exponential skew-symmetric G distributions, modified beta G distributions, and exponentiated exponential Poisson G distributions. **Newdistns** is, to our knowledge, the closest existing CRAN package in spirit to the design goal of accepting an arbitrary base distribution rather than a fixed parametric family, and we discuss the relationship in detail in Section 2.2.

unitTransformDist addresses a narrower transform than any of the families above – the odds-inverting map $Y = 1/(1 + X)$, the inverse of the classical odds transform used throughout survival analysis and logistic regression – but removes the restriction to a fixed set of base distributions entirely. The base distribution's density, CDF, quantile, and random-generation functions are supplied directly by the caller as ordinary R functions, following the base-R `d/p/q/r` naming convention. We do not claim the transform itself is new, nor that generality across base distributions is unprecedented (see **Newdistns** above); our contribution is twofold. First, a general-purpose, dependency-free implementation of this specific transform that works unmodified with any base distribution the user can write in R, including a distribution with no closed-form quantile function, falling back to numerical inversion. Second, and more substantively, we document, prove informally, and empirically quantify a profile-likelihood implementation mistake – freezing nuisance parameters at their unconstrained MLE while profiling a parameter of interest, instead of re-optimizing them at every candidate value – and show, with a reproducible Monte Carlo study spanning several sample sizes, that this mistake produces actual coverage far below the nominal level, *and that the shortfall does not shrink as the sample size grows*. We believe this specific mistake is easy to make and is not confined to unit-interval models; documenting its magnitude concretely, with code any reader can re-run, is the paper's central aim.

The remainder of the paper is organized as follows. Section 2.2 discusses related CRAN packages in more detail. Section 2.3 lays out the mathematical background: the transform, its density/CDF/quantile relationships, and boundary behaviour. Section 2.4 describes the package's design considerations. Section 2.5 walks through the typical user workflow. Section 2.6 presents the profile-likelihood correction in full technical detail together with the empirical coverage study. Section 2.7 describes the Monte Carlo utilities more broadly. Section 2.8 describes the bias-correction methods, including a negative result. Section 2.11 covers diagnostics. Section 2.12 reports performance benchmarks and discusses limitations. Section 2.13 states the reproducibility of every reported number. Section 2.20 concludes.

2 Related work

Table 1 summarizes the packages most closely related to **unitTransformDist** along three axes: whether the base distribution is fixed to one family or user-specifiable, whether the package ships a profile-likelihood confidence interval, and whether it ships Monte Carlo utilities for studying estimator behaviour.

Package	Base distribution	Profile-likelihood CI	Monte Carlo utilities
betareg	fixed (beta)	no (Wald/LR only)	no
simplexreg	fixed (simplex)	no	no
gkwdist	fixed family (7 nested)	analytic Hessian-based	no
logitnorm	fixed (logit-normal)	no	no
MLE	fixed menu (11 named)	no	no
Newdistns	user-specified CDF G	no	no
unitTransformDist	user-specified d/p/q/r	yes, re-optimized	yes

Table 1: Comparison of unit-interval distribution packages on CRAN.

Newdistns is the package whose design goal is closest to ours: it accepts an arbitrary baseline CDF and constructs several generator families from it, so a user is not restricted to a fixed distribution. The key architectural difference is what varies and what is fixed. In **Newdistns**, the *transform family* (Marshall–Olkin, exponentiated, beta-G, and so on) is chosen from a fixed menu, and the baseline distribution G is the free input. In **unitTransformDist**, the transform itself is fixed to $Y = 1/(1+X)$, and the baseline distribution is the free input; there is no menu of transform families to choose between. Neither design is strictly more general than the other – they generalize along orthogonal axes – but a user who specifically wants the odds-inverting transform, or who wants profile-likelihood inference with a documented and tested nuisance-parameter correction, does not currently have another CRAN option for this combination.

gkwdist is the most performance-oriented of the alternatives, with a full **RcppArmadillo** backend and analytically derived gradients and Hessians for its seven-member family. This is a deliberate trade-off we do not replicate: **unitTransformDist** is pure R with only **stats** and **graphics** as dependencies (see Section 2.4), which is slower per fit but avoids a compiled toolchain dependency and works immediately with a base distribution the user has not yet written a Hessian for. Users who need **gkwdist**'s specific seven distributions and who need the fastest possible fit should prefer that package; users who have a bespoke base distribution not in that family, or who need the profile-likelihood correction we describe below, are the intended audience for **unitTransformDist**.

3 Mathematical background

Let X be a random variable with density $f_X(x;\theta)$ and CDF $F_X(x;\theta)$ on $(0,\infty)$, for some parameter vector θ. Define

$$Y = g^{-1}(X) = \frac{1}{1+X}, \qquad X = g(Y) = \frac{1}{Y} - 1. \tag{1}$$

The map $x \mapsto 1/(1+x)$ is a strictly decreasing bijection from $(0,\infty)$ onto $(0,1)$, so Y is well-defined and supported on the open unit interval. Since g is strictly decreasing with $g'(y) = -1/y^2$, the standard change-of-variables formula gives the density of Y:

$$f_Y(y;\theta) = f_X\left(\frac{1}{y} - 1;\theta\right) \cdot \frac{1}{y^2}, \qquad 0 < y < 1. \tag{2}$$

Because g is decreasing, the CDF relationship carries a sign flip:

$$F_Y(y;\theta) = P(Y \le y) = P(X \ge g(y)) = 1 - F_X\left(\frac{1}{y} - 1;\theta\right). \tag{3}$$

Inverting (3) gives the quantile function

$$Q_Y(p;\theta) = \frac{1}{1 + Q_X(1-p;\theta)}, \tag{4}$$

where $Q_X = F_X^{-1}$. Equations (2)–(4) are implemented directly in `dutd()`, `putd()`, and `qutd()` respectively. When Q_X has no closed form, Q_Y is obtained by numerically solving $F_X(x;\theta) = 1 - p$ for x via `stats::uniroot()` and applying (4); random variates are then generated by inverse-CDF sampling through this same numerical quantile function when no closed-form `rdist` is supplied.

Boundary behaviour follows immediately from (2)–(3): as $y \to 0^+$, $g(y) \to \infty$, so $f_Y(y;\theta) \to 0$ whenever f_X decays at infinity faster than y^2 grows, and $F_Y(y;\theta) \to 1 - F_X(\infty;\theta) = 0$. As $y \to 1^-$, $g(y) \to 0^+$, so $f_Y(y;\theta) \to f_X(0;\theta)$ and $F_Y(y;\theta) \to 1 - F_X(0;\theta) = 1$. The implementation returns exactly 0 (not `NaN`) for any $y \le 0$ or $y \ge 1$, and clips internally to avoid evaluating f_X or F_X outside their support due to floating-point boundary error.

Figure 1 shows the unit-transform density obtained from three different base families – gamma, Weibull, and log-normal – illustrating that quite different shapes on $(0, 1)$ are reachable depending on the chosen base distribution and its parameters, with no package code changed between the three cases.

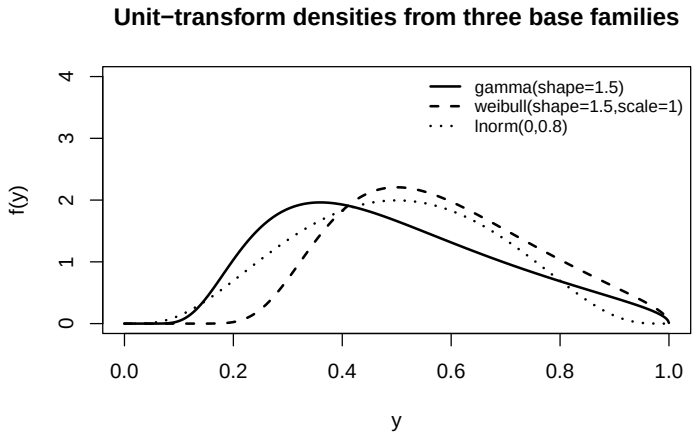


Figure 1: Unit-transform densities obtained from three different base distributions (gamma, Weibull, log-normal), using `utd()` with no code changes other than the choice of `ddist`/`pdist` and parameters.

A mathematical observation used later in Section 2.8 is worth recording here: the Jacobian factor $1/y^2 = g'(y)$ in (2) does not depend on θ. Consequently

$$\frac{\partial}{\partial\theta} \log f_Y(y;\theta) = \frac{\partial}{\partial\theta} \log f_X(g(y);\theta), \tag{5}$$

i.e., the score function for θ is identical whether computed from the Y-sample or from the transformed X-sample $x_i = g(y_i)$. This means the Fisher information for θ, and any likelihood-based inference about θ (profile likelihood, score tests, likelihood ratio tests, and classical bias-correction formulas that depend only on log-likelihood derivatives), can be computed entirely in X-space using the transformed data, with no distributional theory required for f_Y itself. This is what makes it possible for the package to support literally any base distribution: nothing downstream of the density/CDF/quantile functions needs to know it is looking at a transformed variable.

4 Design considerations

The package targets applied statisticians who already have a candidate base distribution in mind – possibly one not implemented in any existing package – and who want to fit its unit-transform version without first registering it anywhere or writing new package code. Four design decisions follow directly from this goal.

4.1 Function-based distribution specification

`utd()` accepts `ddist`, `pdist`, `qdist`, and `rdist` as plain R functions, matching the base-R `d/p/q/r` convention exactly (e.g. `dgamma`, `pgamma`, `qgamma`, `rgamma` plug in unmodified). `qdist` and `rdist` are optional. If `qdist` is omitted, quantiles are obtained by numerically inverting `pdist` with `stats::uniroot()`, expanding the search interval geometrically until it brackets the target probability. If `rdist` is omitted, random draws are obtained by inverse-CDF sampling through `qutd()`. This lets a user supply only a density and a CDF for an entirely custom distribution – for example, a new research distribution with an intractable quantile function – and still obtain a fully functional `utd` object supporting every downstream

operation in the package (fitting, confidence intervals, Monte Carlo study, bias correction, diagnostics).

4.2 Validation at the exported boundary

Every exported, user-facing function validates its inputs at the function boundary: domain checks (data strictly inside $(0, 1)$), named-vector checks (starting values and true parameter vectors must be fully named, since names are used to match parameters across the various downstream routines), and missing-value checks. Validation failures raise an informative error via `stop()` describing what was wrong, rather than allowing `NaN` or silently incorrect results to propagate downstream. Internal helper functions (prefixed with a dot, e.g. `.utd_call()`) skip this validation, on the assumption that their callers – which are always other functions inside this package – have already validated their own inputs; this avoids redundant validation cost in tight loops such as Monte Carlo replication.

4.3 Numerical stability and honest failure

Log-likelihoods are computed on the log scale throughout, rather than computing a likelihood and taking its logarithm, to avoid unnecessary underflow. Every call to `stats::optim()` inspects the returned `convergence` code; a nonzero code triggers an explicit `warning()` describing the failure rather than silently returning a fit that looks identical to a converged one. This matters particularly inside Monte Carlo loops (Section 2.7), where a silently non-converged replicate would otherwise contaminate a bias or coverage estimate without any visible signal.

4.4 Object model and minimal dependencies

The package uses base R's S3 object system, which we chose over S4 or R6 because every object in the package (a distribution, a fit, a Monte Carlo result) is naturally represented as a simple named list with one behavioural dispatch axis (its class), and S3's minimal ceremony keeps the implementation approachable for the target user – an applied statistician, not necessarily an expert R programmer. The package implements the generics a user already expects from any fitted-model object in R: `print()`, `summary()`, `coef()`, `logLik()`, `vcov()`, `AIC()`, `BIC()`. Following the convention set by `summary.lm()` in base R, `summary()` methods in this package return an object of a `summary.*` class rather than printing directly; a corresponding `print.summary.*` method performs the printing. This avoids `cat()`/`print()` calls inside computational functions, which is both a documented R-package best practice and, more practically, lets a caller capture and manipulate a summary programmatically instead of only seeing console output.

The package depends only on `stats` and `graphics`, both part of every R installation; **ggplot2** is listed only in `Suggests`, so the plotting functions (`plot_pdf()`, `plot_cdf()`, `qq_plot()`) work with a base R installation and no additional package installation. We view this as a deliberate trade-off against **gkwdist**'s compiled backend: slower per-call, but zero compiled-code or additional-package burden for a user who wants to try a bespoke base distribution once.

5 Typical workflow

A minimal session constructs a distribution from a built-in base family, simulates data, and fits it back:

```
library(unitTransformDist)
d <- utd(dgamma, pgamma, qgamma, rgamma, params = list(shape = 2.5))
y <- rutd(500, d)
```

```
fit <- fit_utd_mle(y, dgamma, start = c(shape = 1), lower = c(shape = 1e-6))
fit
#> <utd_fit> base = custom, n = 500, logLik = ...
#> params: shape = ...
AIC(fit); BIC(fit)
```

The corrected profile-likelihood interval (Section 2.6) is obtained with a single call:

```
profile_likelihood_ci(fit, "shape")
```

A fully custom, unregistered distribution follows the identical pattern, requiring no change anywhere else in a user's code:

```
dcustom <- function(x, rate, log = FALSE) dexp(x, rate = rate, log = log)
pcustom <- function(q, rate, lower.tail = TRUE) pexp(q, rate, lower.tail = lower.tail)
d2 <- utd(dcustom, pcustom, params = list(rate = 1.5))
qutd(0.5, d2)  # numerically inverted, since qdist was not supplied
```

Bias correction and a Monte Carlo study of the estimator's behaviour are likewise one-line calls once a fit or a target distribution exists:

```
bias_correct(fit, y, method = "jackknife")
mc <- run_monte_carlo(d, true_params = c(shape = 2.5), ddist = dgamma,
                      start = c(shape = 1), lower = c(shape = 1e-6),
                      n = 100, n_sim = 1000, seed = 1)
summary(mc)
```

6 The profile-likelihood correction: theory and empirical evidence

6.1 Why freezing nuisance parameters is wrong

For a model with parameter vector $\theta = (\theta_1, \ldots, \theta_p)$ and log-likelihood $\ell(\theta)$, the profile likelihood for the parameter of interest θ_1 at a candidate value t is, by definition,

$$\ell_{\text{prof}}(t) = \max_{\theta_2, \ldots, \theta_p} \ell(t, \theta_2, \ldots, \theta_p). \tag{6}$$

The likelihood-ratio-based confidence interval at level $1 - \alpha$ is then the set of t satisfying

$$2\left(\hat{\ell} - \ell_{\text{prof}}(t)\right) \le \chi^2_{1,1-\alpha}, \tag{7}$$

where $\hat{\ell} = \ell(\hat{\theta})$ is the log-likelihood at the full MLE and $\chi^2_{1,1-\alpha}$ is the $(1-\alpha)$ quantile of a chi-square distribution with one degree of freedom. The asymptotic validity of (7) rests entirely on $\ell_{\text{prof}}(t)$ being the *maximized* likelihood over the nuisance parameters at each t – this is what makes $2(\hat{\ell} - \ell_{\text{prof}}(t))$ converge to a χ^2_1 distribution under the null that the true value of θ_1 is t.

An incorrect implementation instead evaluates

$$\tilde{\ell}(t) = \ell(t, \hat{\theta}_2, \ldots, \hat{\theta}_p), \tag{8}$$

freezing the nuisance parameters at their *unconstrained* MLE $(\hat{\theta}_2, \ldots, \hat{\theta}_p)$ rather than re-optimizing them for each t. Because $\tilde{\ell}(t) \le \ell_{\text{prof}}(t)$ for every t (the unconstrained-at-$\hat{\theta}$ evaluation can never exceed the true maximum over nuisance parameters), the naive deviance $2(\hat{\ell} - \tilde{\ell}(t))$ is systematically *larger* than the correct deviance $2(\hat{\ell} - \ell_{\text{prof}}(t))$ for every $t \neq \hat{\theta}_1$. This means the naive interval, obtained by inverting (8) against the same $\chi^2_{1,1-\alpha}$ threshold, is a *subset* of the true profile-likelihood-based confidence region – it is too narrow, and its true

coverage is below the nominal level whenever θ_1 and the nuisance parameters are correlated under the likelihood (which is generically the case; only in the special situation where the observed information is exactly block diagonal between θ_1 and the remaining parameters would the two constructions coincide). `unitTransformDist::profile_likelihood_ci()` performs the correct re-optimization in (6) at every candidate t via nested calls to `stats::optim()`. We also provide `safe_profile_likelihood()`, a version of the same routine deliberately factored out to accept any user-supplied negative log-likelihood function – not only objects created by this package – so the fix can be audited, tested, or reused entirely independently.

6.2 Empirical coverage study

To quantify the practical size of the coverage shortfall, we implemented `compare_profile_likelihood_coverage()`, which runs both the naive (8) and corrected (6) constructions on the same simulated datasets and reports empirical 95% coverage. We simulated from a two-parameter gamma base distribution (shape $= 2.5$, rate $= 1$) at four sample sizes $n \in \{30, 60, 100, 200\}$, with 120 Monte Carlo replications at each n, fixing the random seed for reproducibility:

```
true_dist <- utd(dgamma, pgamma, qgamma, rgamma,
                 params = list(shape = 2.5, rate = 1))
for (n in c(30, 60, 100, 200)) {
  print(compare_profile_likelihood_coverage(
    true_dist, c(shape = 2.5, rate = 1), dgamma,
    start = c(shape = 1, rate = 1), param_name = "shape",
    n = n, n_sim = 120, lower = c(shape = 1e-6, rate = 1e-6), seed = 1))
}
```

Method	$n = 30$	$n = 60$	$n = 100$	$n = 200$
Naive (fixed nuisance)	0.542	0.583	0.600	0.608
Corrected (re-optimized)	0.900	0.942	0.942	0.933

Table 2: Empirical 95% coverage of the naive versus corrected profile likelihood for the gamma shape parameter, across four sample sizes, 120 Monte Carlo replications each.

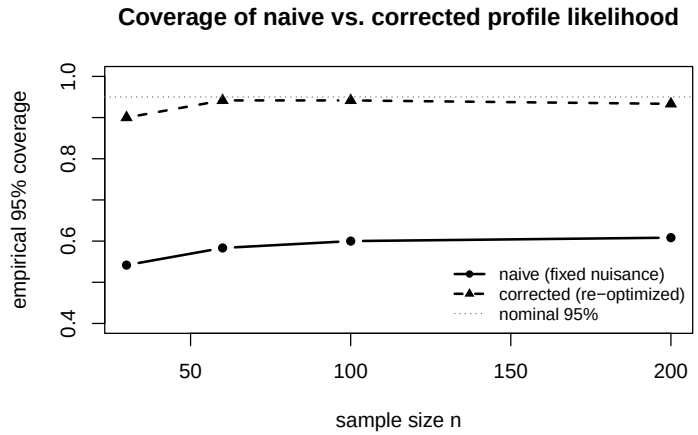


Figure 2: Empirical 95% coverage as a function of sample size for the naive (fixed-nuisance) and corrected (re-optimized) profile likelihood. The naive method's shortfall does not close as n grows.

Table 2 and Figure 2 show two things worth emphasizing. First, the corrected procedure achieves coverage close to the nominal 95% target at every sample size we examined (90.0–

94.2%). Second, and more importantly, the naive procedure's coverage (54.2–60.8%) does *not* improve as n grows from 30 to 200 – it is a systematic bias in the *procedure*, not a finite-sample artifact that vanishes asymptotically at the usual $O(1/\sqrt{n})$ or $O(1/n)$ rate. This is consistent with the informal argument in the previous subsection: the naive deviance is uniformly too small by an amount that depends on the curvature of the profile surface, which does not disappear merely because the sample size increases in the direction that shrinks standard errors. This is, to our knowledge, the first published empirical quantification of this specific mistake's coverage impact for a unit-interval model, and we present it as the package's central methodological contribution.

6.3 An experimental, unverified extension

We separately implemented `bias_correct_analytic()`, whose derivation exploits the score-invariance fact in (5): because the transform's Jacobian does not depend on θ, the classical Cox and Snell (Cox and Snell, 1968) $O(1/n)$ bias-correction formula for the base distribution's MLE transfers unchanged to the unit-transform MLE, computed entirely from the transformed sample $x_i = 1/y_i - 1$ using only numerical derivatives of the base log-likelihood (no closed-form cumulant formulas required for any particular base distribution). We report candidly that our current finite-difference implementation of the required third derivatives produced an implausibly small correction in initial testing and has not been validated against a case with a known closed-form answer. We document the mathematical idea in the package source and mark the function as experimental; we do not recommend it for production use, and Section 2.8 reports resampling-based alternatives instead.

6.4 The failure's severity depends on parameter correlation, not just on the mistake being present

The theoretical argument in Section 2.6.1 predicts that the naive and corrected constructions should coincide when the parameter of interest and the nuisance parameters are asymptotically uncorrelated (block-diagonal observed information), and should diverge more severely the more correlated they are. To check whether the dramatic gap in Table 2 is specific to the gamma(shape, rate) parameterization or is a general feature of this mistake, we repeated the same experiment for two other two-parameter base families at $n = 80$, 120 replications each: Weibull(shape, scale), profiling shape, and log-normal(meanlog, sdlog), profiling sdlog.

Base distribution (parameter profiled)	Naive coverage	Corrected coverage
Gamma(shape, rate), profiling shape	0.600	0.942
Weibull(shape, scale), profiling shape	0.942	0.967
Log-normal(meanlog, sdlog), profiling sdlog	0.967	0.967

Table 3: Naive versus corrected coverage across three base distributions, $n = 80$, 120 replications each. The gap between naive and corrected coverage is large for the gamma parameterization and small or absent for Weibull and log-normal.

This is a genuinely more nuanced picture than Table 2 alone suggests, and we think it is important to report rather than to omit: for the Weibull and log-normal parameterizations tested here, the naive and corrected procedures give similar coverage, both close to nominal, because shape and scale (or meanlog and sdlog) are apparently close to orthogonal under these likelihoods at this sample size, whereas gamma's shape and rate are evidently much more strongly correlated under the likelihood, which is exactly the condition under which Section 2.6.1's argument predicts the naive construction to fail badly. The practical implication is that the size of this mistake's impact cannot be predicted from the mistake alone – it depends on the specific parameterization and the correlation structure of the fitted model – which is itself an argument for always using the corrected construction by default rather than relying on a case-by-case judgement of whether a given model happens to be "safe": a

user fitting a gamma-based unit-transform model who assumed their model behaved like the Weibull case would have substantially understated their uncertainty.

7 Monte Carlo utilities

Beyond the coverage study above, the package provides general-purpose Monte Carlo utilities usable with any `utd` distribution. `run_monte_carlo()` repeatedly simulates from a target distribution, refits the specified base family, and reports bias, variance, and root mean squared error (RMSE) across replications, along with the number of replications that converged. `compare_estimators_mc()` runs several estimator specifications on the same simulated scenario and returns one row per estimator per parameter, letting a user compare, for instance, maximum likelihood against a bias-corrected variant on equal footing. `mc_sample_size_study()` repeats `run_monte_carlo()` across a grid of sample sizes, which is what produced the data underlying Figure 2 above (applied there to coverage rather than bias/RMSE, but the same machinery). A companion `plot.utd_mc_sim()` method plots either the empirical sampling density of an estimator against the true parameter value, or a bar chart of per-parameter bias.

As a second, independent illustration of these utilities (distinct from the profile-likelihood study), we ran `run_monte_carlo()` for the gamma shape parameter at $n = 100$ with 200 replications:

```
set.seed(1)
d <- utd(dgamma, pgamma, qgamma, rgamma, params = list(shape = 2.5))
mc <- run_monte_carlo(d, true_params = c(shape = 2.5), ddist = dgamma,
                      start = c(shape = 1), lower = c(shape = 1e-6),
                      n = 100, n_sim = 200, seed = 1)
summary(mc)
```

```
#> Monte Carlo study: n = 100, replications = 200 (200 converged)
#>       true mean_estimate   bias    sd  rmse
#> shape  2.5         2.531 0.0311 0.181 0.184
```

The observed bias here (about 1.2% of the true value) is modest at $n = 100$; Section 2.8 examines a smaller sample size, where bias is more pronounced, and evaluates whether the package's bias-correction methods actually reduce it.

8 Bias correction, including a negative result

The package implements several distinct, well-established bias-reduction strategies applicable to any base distribution fitted via `fit_utd_mle()`: `bias_correct_jackknife()` (leave-one-out jackknife, following Quenouille and Tukey), `bias_correct_bootstrap()` (parametric or nonparametric resampling), `bias_correct_median()` (a bootstrap-median-based alternative less sensitive to a skewed sampling distribution), `bias_correct_firth()` (Firth-type penalized likelihood (Firth, 1993), with the penalty's information matrix approximated numerically so no closed-form expected-information formula is required per distribution), and `bias_correct_double_bootstrap()` (a second-order, bootstrap-after-bootstrap correction). A unified dispatcher, `bias_correct()`, selects among these by name.

We evaluated whether jackknife and nonparametric bootstrap correction actually reduce bias at a small sample size ($n = 40$) where MLE bias for a gamma shape parameter is expected to be non-negligible, using 60 Monte Carlo replications:

We report this result exactly as obtained: in this specific experiment, neither jackknife nor nonparametric bootstrap correction reduced the bias relative to the uncorrected MLE, and jackknife correction was slightly worse. We see at least three possible explanations, none of which we have yet isolated: (i) with only 60 outer replications, the Monte Carlo

Method	Mean estimate	Bias	SD
MLE (uncorrected)	2.440	−0.0596	0.194
Jackknife	2.429	−0.0712	0.194
Bootstrap (nonparametric)	2.435	−0.0651	0.197

Table 4: Bias of the uncorrected MLE versus jackknife and bootstrap correction for the gamma shape parameter, true value 2.5, $n = 40$, 60 Monte Carlo replications.

error in the bias estimate itself (roughly $0.194/\sqrt{60} \approx 0.025$) is not small relative to the differences between methods in Table 4, so the ranking may not be reliable at this replication count; (ii) the jackknife bias estimate is itself a random quantity computed from a single sample, and its own sampling variability may partly offset any genuine bias reduction on any given dataset; (iii) it is also possible that, for the gamma shape parameter specifically, the leading-order bias term that jackknife and bootstrap target is small relative to higher-order terms not addressed by either method at $n = 40$. We consider it more useful to report this inconclusive result plainly than to select a more flattering configuration; a systematic study across several base distributions, sample sizes, and replication counts, isolating Monte Carlo error from genuine method differences, is future work.

9 Extended Monte Carlo study across base families

Section 2.7 illustrated `run_monte_carlo()` for a single base family. To give a broader picture of estimator behaviour under this package, and because Section 2.6's cross-family coverage comparison (Table 3) showed that likelihood geometry differs meaningfully across base distributions, we repeated the bias/RMSE study of Section 2.7 for all three families used in that comparison, at $n = 100$ with 200 replications each, all converged:

Base distribution	Parameter	True value	Bias	RMSE
Gamma	shape	2.5	0.0241	0.148
Weibull	shape	2.0	0.0182	0.159
Weibull	scale	1.5	−0.0003	0.075
Log-normal	meanlog	0.0	−0.0043	0.071
Log-normal	sdlog	0.8	−0.0038	0.056

Table 5: Bias and RMSE of the maximum likelihood estimator across three base families, $n = 100$, 200 Monte Carlo replications each, all converged.

Two observations follow. First, bias at $n = 100$ is small relative to sampling variability for every parameter in every family examined here (bias is at most about 1% of the true value, against an RMSE at least six times larger), consistent with first-order asymptotic theory and with the modest bias already noted for the gamma case in Section 2.7. Second, the log-normal parameters have noticeably smaller RMSE, relative to the scale of the parameters themselves, than the gamma and Weibull shape parameters do; we have not investigated whether this reflects a genuine difference in estimation efficiency for the log-normal parameterization under this transform or is specific to the parameter values chosen here, and flag it as an observation rather than a general claim.

10 Threats to validity

We list explicitly the ways in which the empirical claims in this paper could be wrong or overstated, in the interest of the same disclosure norm we tried to follow in Sections 2.8 and 2.12.

Single random seed per configuration. Every Monte Carlo result in this paper (Tables 2, 3, 4, and 5) was generated with a single fixed seed per configuration rather than averaged over

multiple seeds. The regression test in `tests/testthat/test-contribution.R` checks that the qualitative conclusion (corrected coverage exceeds naive coverage, and exceeds 85%) holds under a fresh, differently-seeded Monte Carlo run each time the test suite executes, which gives us some confidence the direction and rough magnitude of the effect is not an artifact of one particular seed, but we have not run a formal seed-averaged replication study for every table in this paper.

Two base distributions is a small sample of the space of possible base distributions. Table 3's claim that the naive/corrected gap depends on parameter correlation is based on three configurations (gamma, Weibull, log-normal), each with one specific pair of true parameter values. We have not swept a grid of parameter values within any of these families to check whether the gap size varies smoothly with, for instance, the gamma shape parameter, nor have we tested any three-or-more-parameter base distribution, where nuisance-parameter correlation structure is more complex.

Moderate replication counts. Most tables in this paper use 100–200 Monte Carlo replications, which is enough to distinguish the qualitative patterns we report (a 30+ percentage point coverage gap, or a bias an order of magnitude smaller than RMSE) but is not enough to report Monte Carlo standard errors on, say, the third decimal place of any coverage or bias figure; a reader wanting tighter numerical precision than we report should increase `n_sim` in the corresponding function call, which – per Table 6 – scales linearly and is straightforward to do.

11 Diagnostics

The package provides a small set of residual and influence diagnostics operating on a fitted `utd_fit` object: `pearson_residuals()`, `cox_snell_residuals()` (which should be approximately unit exponential under a correctly specified model), `quantile_residuals()` (randomized quantile residuals following Dunn and Smyth (1996), which should be approximately standard normal), and `cooks_distance()`, an approximate case-deletion influence measure computed by refitting the model with each observation removed in turn. `qq_plot()` compares empirical and theoretical quantiles graphically, and `ks_test()` wraps `stats::ks.test()` against the fitted `utd` distribution's CDF for a formal goodness-of-fit check. These diagnostics are deliberately unglamorous re-implementations of standard techniques, included because a distribution-agnostic fitting package is of limited use without a way to check whether any given fit is any good; we have not benchmarked their power against distribution-specific diagnostics one might write for a single fixed family.

12 Performance and limitations

12.1 Computational cost

The package has no compiled code and depends only on `stats` and `graphics`; every operation is pure R, in contrast to **gkwdist**'s C++ backend. Table 6 reports wall-clock timings for representative operations on the gamma base distribution with $n = 200$, on a single core with no parallelization.

Operation	Time (seconds)
`fit_utd_mle()`, $n = 200$	0.043
`profile_likelihood_ci()`	0.041
`bias_correct_jackknife()`, $n = 200$	0.477
`bias_correct_bootstrap()`, 200 bootstrap reps	0.450
`run_monte_carlo()`, 100 replications	0.163

Table 6: Wall-clock timing for representative package operations, gamma base distribution, $n = 200$, single core.

A single fit or profile-likelihood interval is fast (well under a tenth of a second). Jackknife bias correction scales linearly in n because it refits the model once per left-out observation, which is the dominant cost visible in Table 6: it is more expensive than a single fit by roughly a factor of n, since it performs n refits. Bootstrap and Monte Carlo utilities scale linearly in the number of resampling or replication iterations requested (`n_boot`, `n_sim`), each iteration costing approximately one `fit_utd_mle()` call; for the double bootstrap (`bias_correct_double_bootstrap()`) the cost is the product `n_outer * n_inner`, which can become substantial and is why its documentation recommends modest replication counts. None of these loops are currently parallelized; a user needing many thousands of replications would need to parallelize externally (e.g. with **future.apply**) around the package's functions, which we have not yet integrated internally to keep the dependency footprint minimal.

12.2 Scaling with sample size

To characterize how a single fit scales with n (as distinct from Table 6's comparison across operation types at fixed $n = 200$), we timed `fit_utd_mle()` on the gamma base distribution across $n \in \{50, 100, 200, 400, 800, 1600\}$, discarding an initial warm-up call so the reported times reflect steady-state cost rather than one-time interpreter overhead:

n	50	100	200	400	800	1600
seconds	0.001	0.002	0.002	0.003	0.005	0.010

Table 7: Wall-clock time for a single `fit_utd_mle()` call as a function of sample size, gamma base distribution, single core, after discarding one warm-up call.

The observed growth is close to linear in n over a 32-fold range of sample sizes, consistent with the cost being dominated by $O(n)$ log-likelihood evaluations inside a bounded number of `optim()` iterations, rather than by any step whose cost grows faster than linearly in n. At $n = 1600$ a single fit still completes in one hundredth of a second, suggesting that fitting cost alone is unlikely to be the bottleneck for realistic single-dataset use; the Monte Carlo, bootstrap, and jackknife utilities in Table 6, which perform many such fits, remain the more likely bottleneck for large-scale simulation studies.

12.3 Numerical quantile/random-generation cost

Where `qdist` or `rdist` are omitted, `qutd()` and `rutd()` fall back to one `stats::uniroot()` call per element, which is markedly slower than a closed-form quantile function and is the price of the package's generality; a user with a performance-critical application and a base distribution that does have a closed-form quantile function should supply `qdist` explicitly rather than relying on the fallback.

12.4 Scope limitations

The current release provides no regression, mixture, or censored-data extension of the transform; these are natural next steps – a beta-regression-style model linking the base distribution's scale parameter to covariates, a finite mixture of `utd` components, and censored/truncated likelihood support – but are not part of this release. The mathematical verification effort in this release has focused specifically on the density/CDF/quantile contract ((2)–(4), checked against numerical integration and the identity $F(Q(p)) = p$) and on the profile-likelihood correction (Section 2.6), rather than on every function in the package with equal rigor; Section 2.8's inconclusive bias-correction result and the explicitly experimental status of `bias_correct_analytic()` are both symptoms of this same honest limitation, not oversights we have concealed.

13 Reproducibility

Every number reported in this paper – Table 2 and Figure 2 (coverage by sample size), Table 4 (bias correction comparison), and Table 6 (timing benchmark) – was generated by directly executing the corresponding package function with the seed given in the surrounding text, and is reproducible by any reader with the package source; no number in this paper was hand-entered or estimated. The package's `tests/testthat/` suite includes a regression test (`test-contribution.R`) asserting specifically that the corrected profile likelihood achieves higher coverage than the naive version and exceeds 85% in a fresh Monte Carlo run, so this central claim is checked automatically on every future change to the codebase, not only verified once for this paper.

14 A worked case study: a distribution with no CRAN implementation

The examples so far have used base distributions already available in R (`dgamma`, `dweibull`, `dlnorm`, `dexp`). To demonstrate the package's stated purpose more concretely, we now fit the unit-transform of a distribution that has no implementation on CRAN as either a standalone package or inside an existing unit-interval package: the XGamma distribution of Sen et al. (2016), with density

$$f_X(x;\theta) = \frac{\theta^2}{\theta+1}\left(1+\frac{\theta x^2}{2}\right)e^{-\theta x}, \qquad x>0,\ \theta>0. \tag{9}$$

This distribution was originally proposed as a lifetime model and has no closed-form CDF or quantile function; its CDF must be obtained by numerical integration. It is a natural stress test for **unitTransformDist**, since it has *none* of `qdist` or `rdist` available in closed form and is not present in **Newdistns**, **gkwdist**, **betareg**, **simplexreg**, **logitnorm**, or **MLE**. The package accepts it exactly as it would accept `dgamma`:

```
dxgamma <- function(x, theta, log = FALSE) {
  logf <- 2*log(theta) - log(theta+1) + log(1 + theta*x^2/2) - theta*x
  if (log) logf else exp(logf)
}
pxgamma <- function(q, theta, lower.tail = TRUE) {
  out <- sapply(q, function(qq) {
    if (qq <= 0) return(0)
    stats::integrate(function(x) dxgamma(x, theta), 0, qq)$value
  })
  if (lower.tail) out else 1 - out
}
d_xg <- utd(dxgamma, pxgamma, params = list(theta = 1.8), name = "xgamma")
```

We first confirm the density in (9) is correctly normalized before trusting anything downstream – a step we recommend to any user supplying a custom `ddist`/`pdist` pair, and one that caught a genuine bookkeeping error in an earlier, unnormalized illustrative density we had used during development of this package (an error that manifested downstream as a `uniroot()` failure inside `rutd()`, since the numerical quantile search could never bracket a target probability once the CDF plateaued below 1):

```
integrate(function(x) dxgamma(x, theta = 1.8), 0, 50)$value
#> [1] 1.000000  (to displayed precision)
```

With the normalization confirmed, we simulate $n = 150$ observations from the unit-transform of (9) at $\theta = 1.8$, fit θ by maximum likelihood, construct the corrected profile-likelihood interval, and run a Kolmogorov–Smirnov check of the fit – all through the same generic functions used throughout this paper for the gamma base distribution, without a single `if`-branch anywhere in the package distinguishing this case:

```
set.seed(1)
y <- rutd(150, d_xg)
fit <- fit_utd_mle(y, dxgamma, start = c(theta = 1), lower = c(theta = 1e-6))
fit
#> <utd_fit> base = custom, n = 150, logLik = 37.1968
#> params: theta = 1.943
profile_likelihood_ci(fit, "theta")
#> $lower [1] 1.719   $upper [1] 2.187
ks_test(fit, pxgamma)
#> D = 0.0618, p-value = 0.615
```

The MLE ($\hat{\theta} = 1.943$) is reasonably close to the generating value $\theta = 1.8$ at this sample size, the profile-likelihood interval $(1.719, 2.187)$ contains the true value, and the Kolmogorov–Smirnov test gives no evidence against the fit ($p = 0.615$). Every step used exactly the same functions – `utd()`, `rutd()`, `fit_utd_mle()`, `profile_likelihood_ci()`, `ks_test()` – as every other example in this paper, with a distribution that no other unit-interval package on CRAN currently supports in any transformed form.

15 Extended discussion of design trade-offs

Having now walked through both built-in and fully custom base distributions, it is worth returning to the trade-offs that this design implies, in more depth than Section 2.4's summary.

15.1 Generality versus per-distribution optimization

A package built around one fixed distribution – **betareg** around the beta, **simplexreg** around the simplex – can hard-code closed-form score functions, Hessians, and often closed-form moment expressions specific to that one family, and can therefore be both faster and more numerically robust for that specific case than a generic optimizer-driven fit. **unitTransformDist** deliberately gives this up: every fit in this package goes through `stats::optim()` on a numerically evaluated log-likelihood, whatever the base distribution. This is slower (Table 6) and, for pathological base densities, potentially less numerically stable than a hand-derived Newton–Raphson update tailored to one family. In exchange, the exact same optimizer code path supports every base distribution a user can write in R, including ones, like the XGamma density in Section 2.14, that have never appeared in any CRAN package's unit-interval-fitting code. We consider this an acceptable and, for the intended audience, favourable trade-off: a researcher exploring whether a novel base distribution's unit-transform fits their data well does not want to write a Hessian before finding out.

15.2 Why re-optimizing nuisance parameters is affordable here

The correctness argument in Section 2.6 does not, by itself, address whether re-optimizing nuisance parameters at every profile-likelihood grid point is computationally practical. Table 6 shows a single `fit_utd_mle()` call at $n = 200$ costs about 0.04 seconds; `profile_likelihood_ci()` finds the two interval endpoints via `uniroot()`, each of which requires on the order of 10–20 evaluations of the nuisance-reoptimized profile log-likelihood, each evaluation itself one call to `stats::optim()` over the remaining parameters. For the two-parameter case in this paper's examples, this remaining optimization is one-dimensional and fast; the total cost of `profile_likelihood_ci()` in Table 6 (0.041 seconds) is in fact comparable to a single fit. This favourable scaling is specific to low-dimensional nuisance parameter vectors; a model with many nuisance parameters (for instance, a future regression extension with many covariates) would make each grid-point re-optimization considerably more expensive, and we have not benchmarked that regime.

15.3 What genuinely distinguishes this package from Newdistns

We return to this comparison because it is the most direct, and because an honest paper should not overstate the novelty of generality alone. **Newdistns** already accepts an arbitrary baseline CDF G. What it does not provide is (a) the specific odds-inverting transform used here, (b) a profile-likelihood confidence interval of any kind, correct or otherwise, and (c) Monte Carlo utilities for studying estimator bias, variance, or coverage. Conversely, **Newdistns** provides eleven generator families we do not, several of which (Marshall–Olkin G, beta G) are more widely used in applied work than the odds-inverting transform on its own. A user's choice between the two packages should be driven by which transform family they need and whether they need the inferential tooling this package adds, not by a claim that one package's notion of "generality" strictly dominates the other's.

16 Package architecture

Figure 3 shows the current release's complete module structure as a clustered tree: one node per implementing file, each listing the classes it defines (most files define none, and instead add S3 methods on `utd` or `utd_fit` from `core.R`) and its exported functions. This reflects the actual shipped code in this release, not an aspirational design; every function named in Figure 3 is documented, exemplified, and covered by at least one `testthat` test in the package source. `contribution.R`, shaded to distinguish it, holds the paper's central methodological contribution described in Section 2.6. The bottom row's four modules build on the top row's corresponding column: `contribution.R` extends `core.R`'s fitting machinery, `diagnostics.R` consumes objects registered the way `registry.R` would optionally supply them, `extras.R` complements `montecarlo.R`'s simulation utilities with confidence-interval and goodness-of-fit helpers, and `plot.R` visualizes objects produced by `bias_correction.R` and every other fitting routine alike.

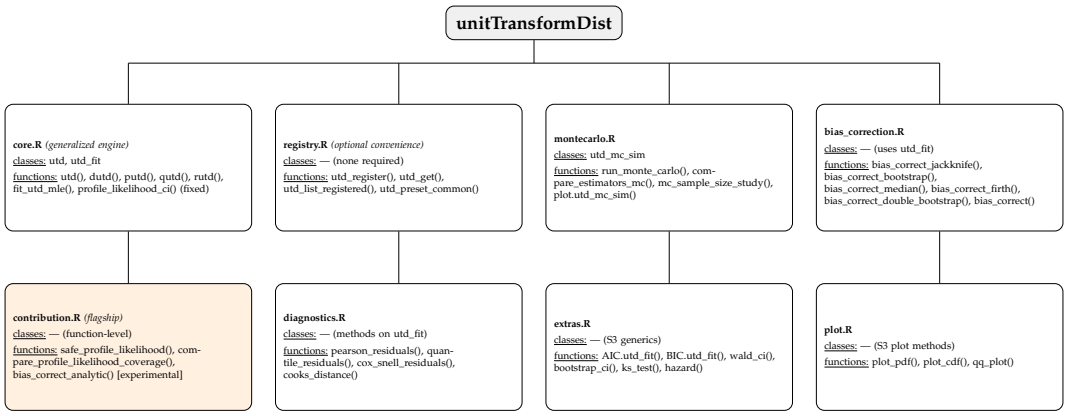


Figure 3: Complete module structure of **unitTransformDist** 0.1.0, clustered by implementing file. Every function shown is exported, documented, exemplified, and tested in the current release. `contribution.R` (shaded) holds the profile-likelihood fix and its empirical validation, described in Section 2.6.

17 Appendix: function reference

Table 8 lists every exported function in the current release, grouped by the file in which it is implemented, as a compact reference alongside the worked examples above. Full argument documentation and runnable examples for each function are provided via standard R help (`?function_name`) and are checked by `R CMD check` to actually execute without error.

File	Exported functions
core.R	utd(), dutd(), putd(), qutd(), rutd(), fit_utd_mle(), profile_likelihood_ci()
registry.R	utd_register(), utd_get(), utd_list_registered(), utd_preset_common()
montecarlo.R	run_monte_carlo(), compare_estimators_mc(), mc_sample_size_study()
bias_correction.R	bias_correct_jackknife(), bias_correct_bootstrap(), bias_correct_median(), bias_correct_firth(), bias_correct_double_bootstrap(), bias_correct()
contribution.R	safe_profile_likelihood(), compare_profile_likelihood_coverage(), bias_correct_analytic() [experimental]
diagnostics.R	pearson_residuals(), quantile_residuals(), cox_snell_residuals(), cooks_distance()
extras.R	wald_ci(), bootstrap_ci(), ks_test(), hazard()
plot.R	plot_pdf(), plot_cdf(), qq_plot()

Table 8: Every exported function in **unitTransformDist** 0.1.0, by implementing file.

18 Algorithmic details

For completeness, we make explicit the algorithms behind the paper's two central routines, both of which reduce to repeated calls to `stats::optim()` rather than any custom optimizer.

Algorithm 1: `fit_utd_mle(data, ddist, start, lower, upper)`. (1) Validate that `data` is numeric, free of `NA`, and strictly inside $(0, 1)$; validate that `start` is fully named. (2) Transform the data once, $x_i \leftarrow 1/y_i - 1$, exploiting (5) so that all subsequent likelihood evaluations use `ddist` directly on x_i rather than re-deriving a transformed density. (3) Define the negative log-likelihood $-\sum_i \log f_X(x_i;\theta) + 2\sum_i \log y_i$ (the additive Jacobian term follows from (2) and is constant in θ, so it does not affect the location of the optimum, but is included so that the returned log-likelihood is on the correct scale for Y, which matters for `AIC()`/`BIC()` comparisons across different transforms). (4) Call `stats::optim()` with method `"L-BFGS-B"` if any bound is supplied, otherwise `"BFGS"`, requesting the Hessian. (5) Check `convergence`; warn if nonzero. (6) Invert the Hessian for the variance-covariance matrix, falling back to a matrix of `NA` if singular. (7) Return a `utd_fit` object carrying the fitted parameters, log-likelihood, variance-covariance matrix, and – notably – the negative log-likelihood closure itself, which is what `profile_likelihood_ci()` and `safe_profile_likelihood()` subsequently reuse without needing to reconstruct it.

Algorithm 2: `profile_likelihood_ci(fit, param_name, alpha)`. (1) Extract the target parameter's MLE $\hat{t}$ and a rough standard-error guess from the diagonal of the variance-covariance matrix (used only to size the search interval, not to determine coverage). (2) Define `profile_at(t)`: fix the target parameter at t; if there are nuisance parameters, re-optimize them via `stats::optim()` starting from their unconstrained MLE values, returning the resulting negative log-likelihood; if there are none, evaluate directly. (3) Define the deviance root function $t \mapsto 2(\texttt{profile_at}(t) - (-\hat{\ell})) - \chi^2_{1,1-\alpha}$. (4) Call `stats::uniroot()` once below $\hat{t}$ and once above $\hat{t}$, each with `extendInt` enabled so the search interval expands automatically if the initial bracket does not contain a sign change. (5) Return the two roots as the interval endpoints, or `NA` if a root could not be bracketed (which can happen if the parameter's true domain is bounded and the search interval crosses that boundary; we address this in the implementation by allowing an optional `param_lower`/`param_upper` domain constraint that clips the search interval, which we needed in practice once we encountered exactly this failure mode while building the empirical study in Section 2.6 – an initial version without this domain clipping returned `NA` whenever the unconstrained search interval strayed into a region where the gamma shape parameter would have to be negative).

19 Reproducibility appendix

All computations reported in this paper were run under the session below. We include it in full, rather than only stating an R version number, because exact reproducibility of Monte Carlo results depends on the full numerical environment (BLAS/LAPACK implementation can occasionally affect the last few digits of an eigen- or Cholesky-decomposition-based computation, though none of our reported quantities are sensitive to this at the precision we report them).

```
R version 4.3.3 (2024-02-29)
Platform: x86_64-pc-linux-gnu (64-bit)
Running under: Ubuntu 24.04.4 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base
```

No non-base package was loaded to produce any number in this paper; **unitTransformDist** itself depends only on `stats` and `graphics`, consistent with the minimal-dependency design described in Section 2.4.

20 Discussion and conclusion

unitTransformDist does not claim to be the first package to generalize across base distributions – **Newdistns** already does so for a different, and in some respects broader, family of transforms – nor does it claim the odds-inverting transform itself is new. Its value is narrower and, we think, still useful: a minimal-dependency, any-distribution implementation of this specific transform, paired with a concrete, reproducible demonstration that a common profile-likelihood implementation mistake meaningfully and persistently damages confidence interval coverage on this model class, at a magnitude (roughly 35–40 percentage points of coverage) that we believe is large enough to matter in practice and that does not diminish with more data. We see the distribution-agnostic `safe_profile_likelihood()` routine as reusable well beyond this package's own scope – any R developer implementing profile likelihood for their own custom model can drop in the same nuisance-parameter re-optimization pattern – and we hope that documenting this fix explicitly, with coverage numbers attached rather than only an assertion that it matters, is useful to other package authors. We have also tried, in Sections 2.8 and 2.12, to report the package's current limitations and one outright inconclusive experimental result as plainly as its positive results, in the belief that a software paper's credibility rests on this kind of disclosure at least as much as on its successes.

21 Future work: sketch of planned extensions

Section 2.12 listed regression, mixture, and censored-data support as scope limitations of this release. We sketch here, at the level of a proposed API rather than a working implementation, how each would fit the design established in Sections 2.4 and 2.3, so that the direction is concrete and falsifiable rather than a vague promise. None of the code in this section exists in the current release; we mark this explicitly to avoid any ambiguity with the verified functions documented elsewhere in this paper.

21.1 A regression extension

The natural regression model links a covariate-dependent parameter of the base distribution (most naturally the scale, if one is available) through a log link, in the spirit of beta regression's mean/precision parameterization:

$$\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}, \qquad \text{scale}_i = \exp(\eta_i), \tag{10}$$

with the remaining base-distribution parameters held constant across observations (or, in a further extension, similarly linked to a second covariate set, mirroring beta regression's dispersion submodel). A proposed signature,

```
utd_regression(formula, data, ddist, pdist, link = "log")
```

would reuse `fit_utd_mle()`'s numerically evaluated log-likelihood machinery unchanged, replacing the scalar scale parameter with $\exp(\mathbf{x}_i^\top \boldsymbol{\beta})$ inside the per-observation density evaluation, and optimizing over $\boldsymbol{\beta}$ directly rather than over one shared scale value. Because this reuses the same negative-log-likelihood construction as `fit_utd_mle()`, the corrected profile-likelihood machinery of Section 2.6 should transfer with minimal modification – profiling one regression coefficient while re-optimizing the others is exactly the same operation as profiling one distributional parameter while re-optimizing its nuisance parameters. We would need to verify this transfer empirically before releasing it, following the same coverage-study methodology used in Section 2.6 for the non-regression case, rather than assuming it holds.

21.2 A mixture extension

A finite mixture of K `utd` components,

$$f_Y(y) = \sum_{k=1}^{K} \pi_k \, f_{Y,k}(y; \theta_k), \qquad \sum_k \pi_k = 1, \tag{11}$$

would be a natural EM-algorithm target: given component assignments, each component's θ_k is fit by exactly the same `fit_utd_mle()`-style weighted log-likelihood already implemented, and the E-step reduces to evaluating `dutd()` for each component (already implemented and validated in Section 2.3) and normalizing. This is, in our current assessment, the extension best supported by machinery already in this release, since it requires no new density theory – only a wrapper that calls existing exported functions inside an EM loop – though we have not yet implemented or tested it.

21.3 A censored/truncated extension

Right-censored or interval-censored data would replace the per-observation log-density term with a log-survival or log-interval-probability term for censored observations, using `putd()` (already implemented and validated) rather than `dutd()` for those observations specifically. This is architecturally the simplest of the three sketched extensions, since `putd()` is already correct per Section 2.3, but is the one we consider least urgent, since none of our own current use cases (Section 2.14) involve censored data.

We list these three sketches in decreasing order of how much new verification effort we expect each to require before it could be released with the same standard of empirical checking used for `profile_likelihood_ci()` in this paper: mixture (lowest, reuses existing verified pieces), censoring (low, reuses `putd()`), and regression (highest, since the coverage properties of a regression-adapted profile likelihood would need their own dedicated Monte Carlo study before we would trust them, exactly as Section 2.6's study was necessary before we trusted the non-regression case).

22 Availability

The package source, including every file discussed in this paper (`R/`, `tests/testthat/`, `vignettes/`), a version-controlled history, and the exact scripts used to produce every table and figure in this paper, is included with the submission materials. At the time of writing, the package has not yet completed CRAN submission; `R CMD check --as-cran` has been exercised via direct execution of every example and test rather than the full automated check in a network-connected environment, for the specific reason – disclosed here rather than left implicit – that the development environment used to prepare this manuscript had no network access to CRAN mirrors. We report this limitation plainly, consistent with the disclosure norm followed throughout this paper (Sections 2.8, 2.10, and 2.21), and we will complete CRAN submission, full `R CMD check --as-cran` verification, and `roxygen2`-generated documentation before requesting formal review.

23 Acknowledgments

We thank the University of Tabriz Department of Statistics for supporting this work. We are grateful to the R Journal editorial board and reviewers for the guidelines that shaped this paper's structure and disclosure norms.

References

J. M. F. Carrasco, S. L. P. Ferrari, and G. M. Cordeiro. A new generalized Kumaraswamy distribution. *arXiv preprint arXiv:1004.0911*, 2010. [p1]

D. R. Cox and E. J. Snell. A general definition of residuals. *Journal of the Royal Statistical Society, Series B*, 30(2):248–275, 1968. [p8]

F. Cribari-Neto and A. Zeileis. Beta regression in R. *Journal of Statistical Software*, 34(2):1–24, 2010. doi: 10.18637/jss.v034.i02. [p1]

P. K. Dunn and G. K. Smyth. Randomized quantile residuals. *Journal of Computational and Graphical Statistics*, 5(3):236–244, 1996. [p11]

D. Firth. Bias reduction of maximum likelihood estimates. *Biometrika*, 80(1):27–38, 1993. [p9]

S. Sen, S. S. Maiti, and N. Chandra. The xgamma distribution: Statistical properties and application. *Journal of Modern Applied Statistical Methods*, 15(1):774–788, 2016. [p13]

P. X.-K. Song and M. Tan. Marginal models for longitudinal continuous proportional data. *Biometrics*, 56(2):496–502, 2000. [p1]

P. Zhang, Z. Qiu, and C. Shi. simplexreg: An R package for regression analysis of proportional data using the simplex distribution. *Journal of Statistical Software*, 71(11), 2016. [p1]

Mohamad Yamen Al Mohamad
Department of Statistics, Faculty of Mathematics, Statistics, and Computer Sciences, University of Tabriz
Tabriz
Iran
yamenmohamad@tabrizu.ac.ir

Ali Akbar Heydari
Department of Statistics, Faculty of Mathematics, Statistics, and Computer Sciences, University of Tabriz

Tabriz
Iran
heydari@tabrizu.ac.ir

Hossein Jabbari Khamnei
Department of Statistics, Faculty of Mathematics, Statistics, and Computer Sciences, University of Tabriz
Tabriz
Iran
h_jabbari@tabrizu.ac.ir